

UNIT — 3 : ARRAYS & COLLECTIONS IN JAVA

Java Programming — Complete Unit 3 Notes

Course: Object-Oriented Programming through Java | **Year/Sem:** 2nd Year Sem 1 (2-1)

Syllabus Coverage: Arrays Introduction, Declaration & Initialization, Memory Storage Model, Operations, Arrays Class, Multidimensional & Jagged Arrays, Three-Dimensional Arrays, and ArrayList vs. Vector.

Source: ChatGPT Curated Exam Preparation Notes

1. Introduction to Arrays

An **array** is a collection of elements of the **same data type** stored under one variable name. Each element is accessed using an **index**.

- **0-indexed:** In Java, array indexing begins at 0 and goes up to length - 1.
- **Fixed Size:** Once allocated in memory, the size of a standard Java array cannot grow or shrink.
- **Object Nature:** Arrays are first-class objects in Java, allocated on the Heap memory.
- **Type Safety:** Can store primitive types (e.g. int, double) or Object references.

Syntax:

```
dataType[] arrayName = new dataType[size];
```

Example Program:

```
int[] marks = {85, 90, 78, 92, 88};  
System.out.println("First element: " + marks[0]);  
System.out.println("Array length: " + marks.length);
```

Output:

```
First element: 85  
Array length: 5
```

Real-Time Application: A college portal storing the exam marks of 5 students in a class.

2. Declaration and Initialization of Arrays

Declaration informs the compiler about the array's type and name without allocating heap memory.

Initialization allocates the memory block and assigns default or specific values.

- **Declaration:** `int[] marks;` (creates reference variable on stack).
- **Memory Allocation:** `marks = new int[5];` (allocates array with default value 0 on heap).
- **Combined Declaration & Allocation:** `int[] marks = new int[5];`
- **Direct Literal Initialization:** `int[] marks = {80, 85, 90, 75, 95};`

Example Program:

```
class ArrayExample {
    public static void main(String[] args) {
        int[] marks = {80, 85, 90, 75, 95};
        for (int i = 0; i < marks.length; i++) {
            System.out.println("Student " + (i + 1) + " Marks = " + marks[i]);
        }
    }
}
```

Output:

```
Student 1 Marks = 80
Student 2 Marks = 85
Student 3 Marks = 90
Student 4 Marks = 75
Student 5 Marks = 95
```

Real-Time Application: Storing student scores or monthly temperature records for automated analysis.

3. Storage of Array in Computer Memory

In Java, arrays are allocated on the **Heap**. The variable on the **Stack** holds a 4-byte or 8-byte reference pointing to the heap memory location. Elements are stored in contiguous memory cells, allowing **O(1) constant-time direct indexing**.

- Contiguous indexed allocation enables instant arithmetic lookup: $\text{address} = \text{base_address} + (\text{index} * \text{data_type_size})$.
- Default values upon heap allocation: Numeric types $\rightarrow 0$, boolean $\rightarrow \text{false}$, references $\rightarrow \text{null}$.

Syntax:

```
int[] a = {10, 20, 30, 40};
```

Conceptual Memory Mapping:

Index	0	1	2	3
Value	10	20	30	40
Address (Simulated)	0x1000	0x1004	0x1008	0x100C

Real-Time Application: Storing railway coach seat numbers or consecutive locker assignments.

4. Accessing Elements of Arrays

Array elements are accessed directly by specifying the index in square brackets [index]. Accessing an index outside [0, length - 1] throws `ArrayIndexOutOfBoundsException`.

Syntax:

```
arrayName[index];
```

Example Program:

```
class AccessArray {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40, 50};
        System.out.println("First element: " + numbers[0]);
        System.out.println("Fourth element: " + numbers[3]);
        System.out.println("Last element: " + numbers[numbers.length - 1]);
    }
}
```

Output:

```
First element: 10
Fourth element: 40
Last element: 50
```

Real-Time Application: Fetching a specific patient's daily temperature reading by day index.

5. Operations on Array Elements

Common operations include: **Traversing** (looping through all items), **Updating** (modifying a value at an index), **Sum & Average** calculation, and finding **Maximum & Minimum** elements.

Example Program:

```
class ArrayOperations {
    public static void main(String[] args) {
        int[] marks = {80, 70, 90, 85, 75};
        int sum = 0, max = marks[0], min = marks[0];

        for (int mark : marks) {
            sum += mark;
            if (mark > max) max = mark;
            if (mark < min) min = mark;
        }

        double average = (double) sum / marks.length;
        System.out.println("Total Sum: " + sum);
        System.out.println("Average: " + average);
        System.out.println("Maximum: " + max + " | Minimum: " + min);
    }
}
```

Output:

```
Total Sum: 400
Average: 80.0
Maximum: 90 | Minimum: 70
```

Real-Time Application: Calculating total bill, average spending, and highest transaction in an accounting system.

6. Assigning One Array to Another Array

Assigning one array variable to another (b = a;) copies only the **reference**, not the underlying array object. Both variables now point to the exact same memory location on the heap. To create a separate, independent duplicate, use `a.clone()` or `System.arraycopy()`.

Example Program:

```
class ArrayAssignment {
    public static void main(String[] args) {
        int[] a = {10, 20, 30};
        int[] b = a;           // Reference copy
        int[] c = a.clone();   // Independent copy

        b[0] = 999;
        System.out.println("a[0] via b modification: " + a[0]); // 999
        System.out.println("c[0] independent clone: " + c[0]);   // 10
    }
}
```

Output:

```
a[0] via b modification: 999  
c[0] independent clone: 10
```

Real-Time Application: Understanding shallow copies vs deep copies in security-critical and transactional data.

7. Dynamic Change of Array Size

Standard Java arrays are **static and fixed in size**. If capacity is exhausted, a programmer must allocate a larger array and copy all elements over. To handle dynamic sizing automatically, Java provides Collections like ArrayList and Vector.

Syntax:

```
ArrayList<Integer> list = new ArrayList<>();
```

Example Program:

```
import java.util.ArrayList;  
  
class DynamicArrayDemo {  
    public static void main(String[] args) {  
        ArrayList<Integer> numbers = new ArrayList<>();  
        numbers.add(10);  
        numbers.add(20);  
        numbers.add(30);  
        numbers.add(40);  
        System.out.println("Dynamic ArrayList: " + numbers);  
        System.out.println("Size: " + numbers.size());  
    }  
}
```

Output:

```
Dynamic ArrayList: [10, 20, 30, 40]  
Size: 4
```

Real-Time Application: An e-commerce shopping cart where products can be added or removed at any time.

8. Sorting of Arrays

Sorting rearranges elements into ascending or descending order. Java provides the optimized Dual-Pivot Quicksort via `Arrays.sort()` in `java.util.Arrays` with **$O(N \log N)$** performance.

Syntax:

```
Arrays.sort(arrayName);
```

Example Program:

```
import java.util.Arrays;  
  
class SortingArray {  
    public static void main(String[] args) {  
        int[] marks = {75, 90, 65, 88, 70};  
        Arrays.sort(marks);  
        System.out.println("Sorted Marks: " + Arrays.toString(marks));  
    }  
}
```

Output:

Sorted Marks: [65, 70, 75, 88, 90]

Real-Time Application: Sorting leaderboard ranks, merit lists, or prices from lowest to highest.

9. Search for Values in Arrays

Searching is checking whether a given value exists in an array and finding its index. **Linear Search** scans elements sequentially ($O(N)$), while **Binary Search** requires a sorted array and executes in logarithmic time ($O(\log N)$).

Example Program:

```
class SearchArray {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40, 50};
        int target = 30;
        int foundIndex = -1;

        for (int i = 0; i < numbers.length; i++) {
            if (numbers[i] == target) {
                foundIndex = i;
                break;
            }
        }
        System.out.println(foundIndex != -1 ? "Found at index: " + foundIndex : "Not found");
    }
}
```

Output:

Found at index: 2

Real-Time Application: Searching for a student roll number, phone number, or employee ID in a database.

10. Class Arrays (java.util.Arrays)

java.util.Arrays is a built-in utility class containing static methods to manipulate, sort, search, copy, and compare arrays easily.

- Arrays.sort(arr): Sorts array in ascending order.
- Arrays.toString(arr): Converts array elements into readable string representation.
- Arrays.copyOf(arr, length): Copies array to new array with specified length.
- Arrays.equals(arr1, arr2): Checks if two arrays have equal elements in the same order.
- Arrays.fill(arr, val): Fills all elements of the array with a specific value.
- Arrays.binarySearch(arr, key): Performs fast binary search on sorted array.

Example Program:

```
import java.util.Arrays;

class ArraysClassDemo {
    public static void main(String[] args) {
        int[] a = {30, 10, 20};
        Arrays.sort(a);
        System.out.println("Sorted: " + Arrays.toString(a));

        int[] b = Arrays.copyOf(a, a.length);
        System.out.println("Arrays equal: " + Arrays.equals(a, b));

        int pos = Arrays.binarySearch(a, 20);
        System.out.println("Element 20 at index: " + pos);
    }
}
```

Output:

```
Sorted: [10, 20, 30]
Arrays equal: true
Element 20 at index: 1
```

Real-Time Application: Standardizing array processing without writing manual sorting and duplication algorithms.

11. Two-Dimensional Arrays (2D Arrays)

A 2D array stores data in tabular format with **rows and columns**. In Java, a 2D array is an array of 1D array references.

Syntax:

```
dataType[][] arrayName = new dataType[rows][columns];
```

Example Program:

```
class TwoDArrayDemo {
    public static void main(String[] args) {
        int[][] matrix = {
            {1, 2, 3},
            {4, 5, 6}
        };

        for (int i = 0; i < matrix.length; i++) {
            for (int j = 0; j < matrix[i].length; j++) {
                System.out.print(matrix[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

Output:

```
1 2 3
4 5 6
```

Real-Time Application: Storing student marks across multiple subjects, image pixel grids, or chess boards.

12. Arrays of Varying Lengths (Jagged Arrays)

A **jagged array** is a multidimensional array where each row can have a **different number of columns**. This saves memory when row sizes are uneven.

Syntax:

```
int[][] a = new int[3][]; // define 3 rows
a[0] = new int[2];        // row 0 has 2 elements
a[1] = new int[3];        // row 1 has 3 elements
a[2] = new int[4];        // row 2 has 4 elements
```

Example Program:

```
class JaggedArrayDemo {
    public static void main(String[] args) {
        int[][] students = {
            {80, 90},
            {70, 75, 85},
            {60, 65, 70, 80}
        };

        for (int i = 0; i < students.length; i++) {
            System.out.print("Student " + (i + 1) + " marks: ");
            for (int j = 0; j < students[i].length; j++) {
                System.out.print(students[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

Output:

```
Student 1 marks: 80 90
Student 2 marks: 70 75 85
Student 3 marks: 60 65 70 80
```

Real-Time Application: Recording completed course credits for students where each student took a different number of courses.

13. Three-Dimensional Arrays (3D Arrays)

A 3D array is an array of 2D arrays, visualized as **layers/blocks** → **rows** → **columns**.

Syntax:

```
int[][][] arr = new int[depth][rows][columns];
```

Example Program:

```
class ThreeDArrayDemo {
    public static void main(String[] args) {
        int[][][] data = {
            { {1, 2}, {3, 4} },
            { {5, 6}, {7, 8} }
        };

        for (int i = 0; i < data.length; i++) {
            System.out.println("Layer " + i + ":");
            for (int j = 0; j < data[i].length; j++) {
                for (int k = 0; k < data[i][j].length; k++) {
                    System.out.print(data[i][j][k] + " ");
                }
                System.out.println();
            }
        }
    }
}
```

Output:

```
Layer 0:
1 2
3 4
Layer 1:
5 6
7 8
```

Real-Time Application: Campus facilities management: Campus Building → Floor Level → Room Sensor Data.

14. Arrays as Vectors and Lists

Java provides powerful dynamic collections that implement dynamic arrays under the hood: Vector and ArrayList.

- **Vector:** Legacy class from Java 1.0. Dynamically resizable and **thread-safe** because its methods are synchronized.
- **ArrayList:** Introduced in Java 1.2. Resizable array implementation of the List interface. **Not synchronized**, making it much faster in single-threaded scenarios.

Example Program:

```
import java.util.Vector;
import java.util.ArrayList;

class VectorAndListDemo {
    public static void main(String[] args) {
        // Vector demonstration
        Vector<Integer> vec = new Vector<>();
        vec.add(10); vec.add(20); vec.add(30);
        System.out.println("Vector elements: " + vec);

        // ArrayList demonstration
        ArrayList<String> list = new ArrayList<>();
        list.add("Ravi"); list.add("Rahul"); list.add("Priya");
        System.out.println("ArrayList initial: " + list);
        list.remove("Rahul");
        System.out.println("ArrayList after removal: " + list);
    }
}
```


Output:

```
Vector elements: [10, 20, 30]
ArrayList initial: [Ravi, Rahul, Priya]
ArrayList after removal: [Ravi, Priya]
```

Real-Time Application: Managing dynamically changing carts, shopping wishlists, or session activity logs.

15. Quick Revision Table

Concept	Key Exam Point	Syntax / Class
Array	Fixed-size, homogeneous elements	<code>int[] a = new int[5];</code>
Declaration	Creates reference on stack	<code>int[] a;</code>
Initialization	Allocates memory on heap	<code>a = new int[5];</code>
Access	Direct 0-based indexing $O(1)$	<code>a[0]</code>
Length	Read-only property (not method)	<code>a.length</code>
Assignment	Copies reference, not elements	<code>b = a;</code>
Dynamic Size	Resizable collections	ArrayList / Vector
Sorting	Dual-pivot quicksort $O(N \log N)$	<code>Arrays.sort(a);</code>
Searching	Linear ($O(N)$) or Binary ($O(\log N)$)	<code>Arrays.binarySearch(a, val);</code>
Utility Class	Helper methods for arrays	<code>java.util.Arrays</code>
2D Array	Array of 1D arrays (rows x cols)	<code>int[][] a = new int[3][4];</code>
Jagged Array	Rows have varying column sizes	<code>int[][] a = new int[3][];</code>
3D Array	Block -> Row -> Column layout	<code>int[][][] a = new int[2][3][4];</code>
Vector	Thread-safe dynamic array (Java 1.0)	<code>Vector<T> v = new Vector<>();</code>
ArrayList	Fast non-synchronized list (Java 1.2)	<code>ArrayList<T> l = new ArrayList<>();</code>

16. High-Frequency Exam Question: Array vs. ArrayList

Feature	Array	ArrayList
Size	Fixed (static) size	Dynamic (resizable) size
Data Types	Primitive types (int, char) & Objects	Objects only (uses Wrappers like Integer)
Syntax	<code>int[] arr = new int[10];</code>	<code>ArrayList<Integer> arr = new ArrayList<>();</code>
Size Determination	<code>arr.length</code> (property)	<code>arr.size()</code> (method)
Performance	Faster (low overhead, no boxing)	Slightly slower (boxing & dynamic resizing)
Generics	Does not support type parameters	Full Generics support (Type-safe)
Addition / Deletion	Manual shifting required	Built-in methods (add, remove, clear)

17. Difference Between ArrayList and Vector in Java

Both **ArrayList** and **Vector** are part of the Java Collections Framework and implement the List interface backed by dynamic arrays. They can grow or shrink automatically as elements are added or removed.

Feature	ArrayList	Vector
Package	<code>java.util</code>	<code>java.util</code>

Feature	ArrayList	Vector
Size Nature	Dynamic / Resizable	Dynamic / Resizable
Synchronization	Not synchronized (Not thread-safe)	Synchronized (Thread-safe)
Performance	Faster (no synchronization lock overhead)	Slower (synchronization overhead)
Capacity Growth	Increases capacity by 50% (new = old * 1.5)	Doubles capacity by 100% (new = old * 2)
Version Introduced	Java 1.2 (Collections Framework)	Java 1.0 (Legacy Class)
Traversal Method	Iterator, ListIterator, for-each	Enumeration, Iterator, ListIterator
Industry Preference	Preferred for almost all standard apps	Rarely used unless legacy compatibility needed

Easy Memory Mnemonic:

ArrayList: *Faster, Light-weight, Not Synchronized (Modern default)*

Vector: *Thread-Safe, Heavier, Synchronized, Doubles Capacity (Legacy 1.0)*

Dynamic Expansion Concept (Shopping Cart Flow):

```
Customer adds products to cart
      ↓
Collection buffer automatically scales capacity
      ↓
Initial capacity (10) --> Expanded (15 / 20) --> Expanded (30 / 40)
      ↓
ArrayList grows by 50% | Vector doubles capacity (100%)
```

Original ChatGPT Conversation Source & Reference:

Shared Chat Link: <https://chatgpt.com/share/6ac3d3a6-7ebc-83e8-9310-36ef53eb3ba0>

Note: This document compiles the complete B.Tech / Degree 2nd Year 1st Semester (2-1) Java Programming notes for Unit-3 (Arrays & Collections), generated directly from the ChatGPT session at the above link.